

게임 개발자 포트폴리오

노훈철

Tel. 010 - 2365 - 6395

Email : nohc2001@naver.com

프로젝트

1. 개발

1. PulseCity (게임공학과 졸업작품)
2. Otornait (윈도우 프로그래밍 기말과제, 교내 과제전 2등 수상)
3. 그래픽스 기말과제 (게임그래픽스 기말과제)

2. 기획

1. GMTK 게임잼 참여
 1. Inverse Proportion
 2. God Choice

Pulse City

프로젝트 이름 : Pulse City

장르 : 오픈월드 FPS 다중접속 RPG

개발인원 : 3명 / 개발 기간 (2025-06-12 ~ 2026-06-22)

개발 환경 :

IDE/Debugger : VisualStudio2022, PIX(GPU 디버깅)

Language : c++20

System API : Windows API, DXGI

Rendering : DirectX 12, hlsl

GameData Making : Unity + C#, Blender

GameData Finding : SketchFab, AmbientPBR, turbosquid, cgtrader

TTF Parsing : ttfParser.h (<https://github.com/kv01/ttf-parser>)

Sound : Window WASAPI

플레이 및 소개영상

<https://youtu.be/Z8GBns70uf0>

내가 담당한 역할 / 수행한 과제 요약

일단 중요했던 과제들을 먼저 나열하고, 6번에서 나머지 소소한 것들을 소개한다.

1. 유니티 바이너리 익스포터 제작
2. 서버 프레임워크 제작
3. 게임 프레임워크 제작
4. 렌더링
5. 레이트레이싱
6. 그 외의 기여
7. 부록

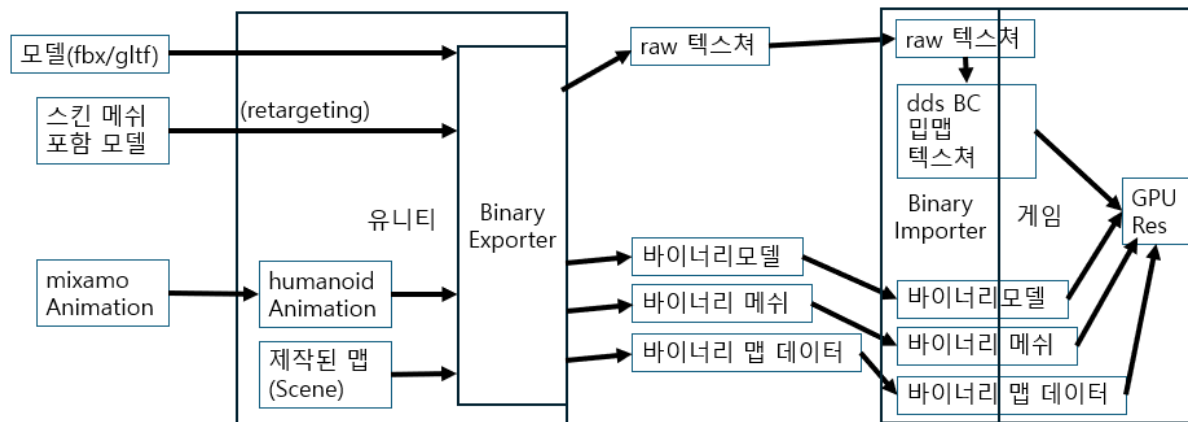
유니티 바이너리 익스포터 제작

유니티와 C# 스크립트를 활용한 [맵 - 모델 - 스킨메쉬 포함 모델 - 애니메이션 바이너리 익스포터] 제작
c++로 바이너리 임포터 제작.

여러 존 맵을 하나의 Scene에서 만들어 볼 수 있고, 그래서 자연스럽게 연결되게 할 수 있다.

익스포터 스크립트뿐 아니라, 익스포트에 걸림돌이 되는 요소들을 정리하는 Editor 스크립트또한 제작.

이름에서 특수문자와 띄어쓰기를 제외하는 스크립트 -> DDS 변환을 위해서
모든 에셋을 Read/Write Enable로 만드는 스크립트 -> 익스포트를 위해서
메쉬의 pivot을 자신의 중심으로 만드는 스크립트 -> 원활한 맵 제작을 위해서
등등..



<게임 데이터를 GPU Resource로 가져오는 흐름도>
애니메이션을 빼먹었는데 애니메이션도 바이너리 익스포트하고 임포트한다.

또한 팀원과의 원활한 소통을 위해
맵 제작 가이드라인을 업데이트하고, 공유했다.

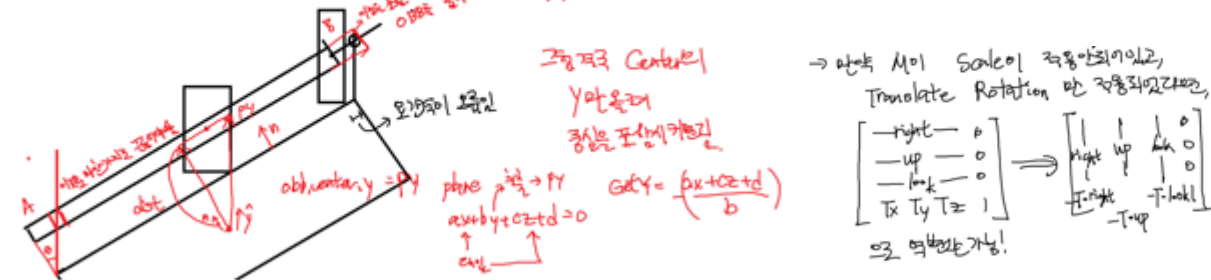


서버 프레임워크 제작

서버 프레임워크의 핵심적 내용. - 서버를 만들긴 했지만 사실 한건 별로 없다.

1.xyz로 이동이 가능하도록 OBB 서버 충돌 계산

서버에서 3D로 움직여야 할 게임 오브젝트간 충돌을 계산하기 위해 많은 고민을 했다. 그리고 특별한 오류없이 AOBB 충돌을 수행하는 서버를 만들었다.



해당 기록의 일부는 <부록 - 서버의 충돌계산 고민 메모>를 참고하자.

3.Recv시에 잘리는 데이터 복구로직

이건 그냥 필수적인 로직이다. 그걸 썼다. 할말이 없다.

4.STC 변수 동기화

게임오브젝트의 동기화할 필요가 있는 모든 멤버를 위해 일일이 프로토콜을 만드는 대신, `ChangeMemberGameObject` 프로토콜 하나로 게임오브젝트의 모든 STC 멤버를 동기화 가능하게 하기.

매크로를 활용해서 만들었다.

STC는 Server To Client의 약자이며, STCDef(type, name); 으로 선언이 가능하다.

STC로 선언된 멤버변수는 ChangeMemberGameObject 프로토콜을 보내서

클라이언트에 있는 같은 멤버를 수정할 수 있다.

2.WSAPoll으로 nonblocking recv, WSASend, SendDataSaver활용해 패킷 사이즈 줄이기

WSAPoll을 활용해 서버는 recv를 받게 된다.

Server의 send는, 게임루프에서 SendDataSaver에 패킷을 먼저 저장해놓고 나중에 send를 하게 되는데, 특정 클라에게만 보내줄 데이터는 PersonalSDS에, 해당 Zone의 모두에게 보내줄 데이터는 CommonSDS에 담는다. CommonSDS에는 한번만 데이터를 담고, 모두가 공유하기 때문에 데이터 복사가 줄어들며, WSASend로 PersonalSDS와 CommonSDS를 한번의 호출로 send한다. 그렇게 되면 데이터를 보낼때마다 한번에 보내게 되어서, 보내는 패킷의 양이 줄어들 것이라 생각하고 설계했다.

프로토콜의 제작법은 게임서버 프로그래밍 과목에서 배운 프로토콜 구성을 그대로 따왔다.

```
> /* ... */  
struct Monster : public SkinMeshGameObject {  
#define STC_CurrentStruct Monster  
STC_STATICINIT innerStruct;  
  
STCDef(float, HP); // ...  
STCDef(float, MaxHP); // = 30;  
STCDef(float, Defense) // ...  
STCDef(bool, isDead);// = false;  
  
MonsterType m_monsterType = MonsterType::Walker;  
float Attack = 10.0f;  
  
vec4 m_homePos;  
vec4 m_targetPos;  
float m_speed = 2.0f;  
float m_patrolRange = 20.0f;  
float m_patrolSpeed = 10.0f;
```

게임 프레임워크 제작

1. 청크시스템과 Static-Dynamic Object 처리

클라이언트에서의 ViewFrustumCulling이나, 보이는 스킨 메쉬만 업데이트하기 위해서 하나의 존을 나누는 청크가 필요했고, 서버쪽에서는 충돌처리에 고려되는 오브젝트 개수를 줄이기 위해 청크가 필요했다. 그래서 청크 시스템을 만들고, Object를 움직이지 않는 StaticObject와 움직이는 DynamicObject로 나누었다. DynamicObject가 청크 위에서 계산되기 위해 구조를 수정했다. 더 자세한 내용은 <부록 - 청크에 들어가기 위한 게임오브젝트 구조 설계>를 참고하자.

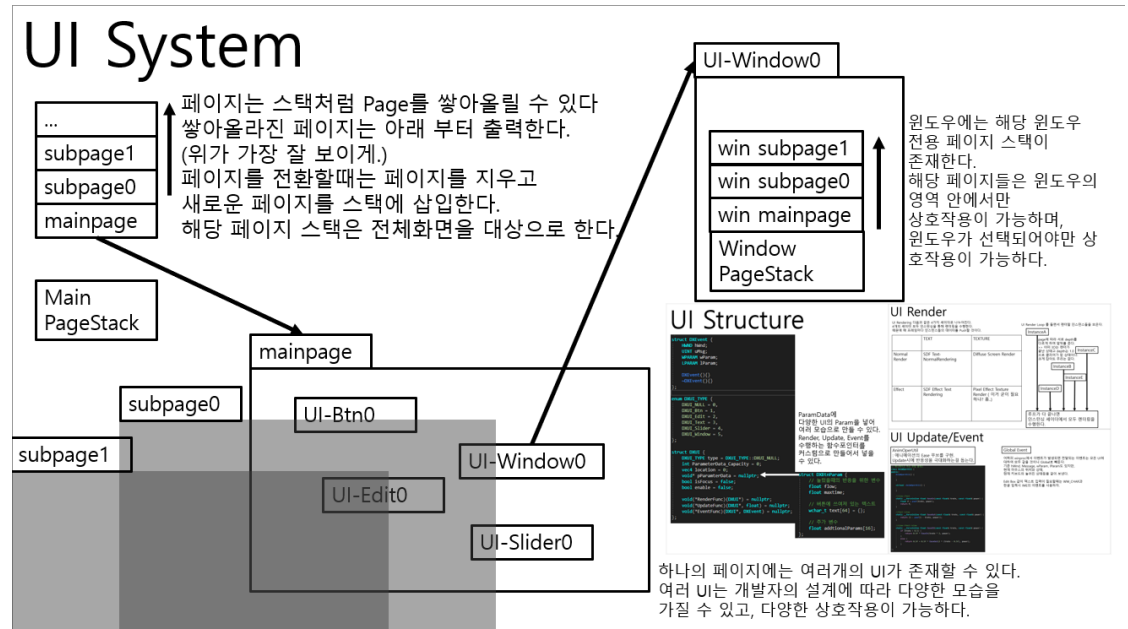
3. UI system 제작

게임에 제대로된 UI가 들어갈 수 있게 하고 싶었다. 총 Window, Slider, Btn, EditBox, Slot UI를 제작했고, Page에 해당 UI들이 포함가능하며, Window는 여러 Page를 포함가능하다. 게임에서 상호작용이 가능.

2. Window WASAPI을 활용한 사운드 시스템 제작

Fmod 같은걸 쓰지 않은 이유는 프로젝트가 너무 무거워 질것 같기도 하고, 또 새로운 라이브러리를 배운다는건 버겁다고 생각되었다. 사운드 작업은 CPU작업이라 내가 통제하고 싶었다. 통제불가한 익명의 스레드에 사운드 작업을 맡기고 싶지는 않았던것 같다. Fmod같은 라이브러리를 쓰지 않은 이유다. 사실 개인적 욕심이고 그럴 필요는 없었다.

마스터 채널과 채널들을 연결할 수 있고, 채널의 볼륨을 줄이거나 높일 수 있다.



렌더링

1. 간단한 PBR 셰이딩 구현

PBR 셰이딩을 LearnOpenGL 문서를 보고 배운후, PBR 셰이더를 제작했다.
첫시도에 잘 되지 않아서 여러번 시도했다.
지금은 메탈릭 맵과 러프니스 맵이 잘 반영되는 모습,
IBL 반사가 되는 모습을 확인할 수 있다.
다만 PBR의 모든 특징을 담아냈다고 보기는 어려울 것 같다.
러프니스가 올라가면 반사가 흐려져야 하는데,
그 부분은 제대로 구현이 안되었다.
또한 EmissionLightMaterial이 처리가 안된다.
추후 수정할 예정이다.

3. 청크를 활용한 뷰 프러스텀 컬링

RenderTour 함수를 통해 선택된 viewport에 대해 프러스텀 컬링을 한 오브젝트를 뽑거나, 렌더를 해서 DrawCall을 줄이고, GPU가 할일을 줄인다.

이는 세도우 맵핑을 할때도 viewport를 직교투영으로 설정하여 오브젝트를 추린다.

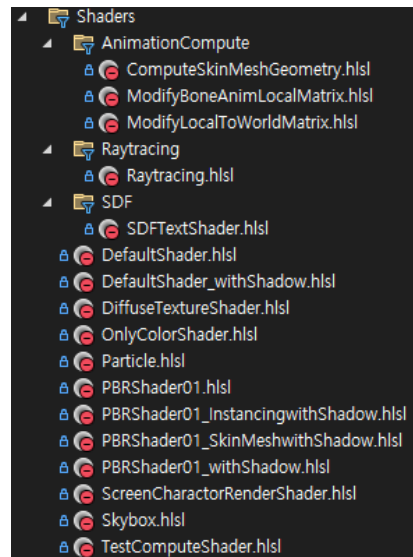
2. Static 다중 라이팅

다중 라이팅이 작동한다. StructuredBuffer를 사용해 모든 청크의 라이트 데이터를 담아서 셰이더에서 위치 정보를 통해 주변의 라이트에 접근할 수 있게 만들었다.
때문에 만약 DynamicLight 또한 여기에 담고 움직이다 라이트 없어질때 해제하면 되서 추후에 구현이 가능해 보인다.
다만 StructuredBuffer를 통해 라이트 데이터를 모두 바인딩 하기 때문에 한 청크당 라이트 수가 제한되어 있는 점은 단점이다. 32개다.
또한 Light는 Point, Spot 두개 종류의 라이트 모두 구현했다.
DirLight 정보는 따로 바인딩을 한다.

4. CascadeShadowMap (3단계)

3단계의 CascadeShadowMapping을 진행한다.

레스터 PBR 셰이더, 스카이박스 셰이더,
모션블러와 거리에 따른 합성곱 수행 셰이더,
SDFText를 출력하기 위한 셰이더와
레이트레이싱 렌더링을 위한 셰이더,
애니메이션 작동을 위한 3개의 셰이더등
렌더의 대부분을 작성함.



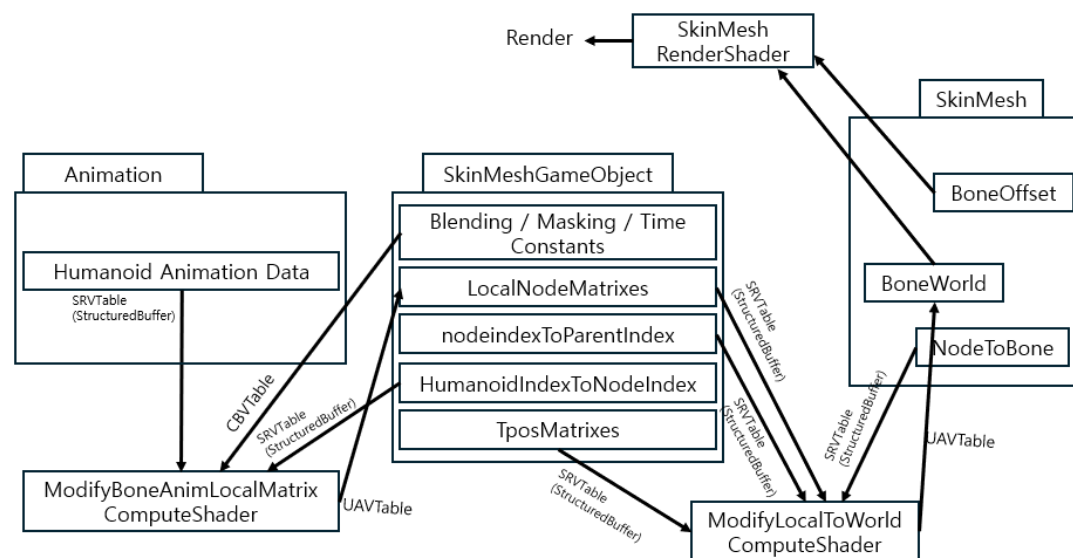
TTF에서 읽어온 텍스트의 데이터를 SDF로 재구성하여 화면에 출력할 수 있도록 한다. SDF 텍스처를 만드는 과정이 픽셀 하나를 찍을 때마다 모든 픽셀에 대해 거리를 계산을 수행해 매우 느렸으나, 원점에서 가장 가까운 픽셀 순서대로 순회할 수 있는 테이블을 만들어 빠르게 새로운 텍스트에 대한 SDF 구성을 할 수 있었다.

글자를 이루는 외곽선을 기준으로 한 픽셀은 외곽선과의 최소거리를 저장한다. 부호는 내부일 경우 음수 외부일 경우 양수가 된다.
이것을 활용해 다양한 글자의 효과를 구현이 가능하다.
기본적으로 출력할 픽셀의 값의 최소값과 최대값을 지정하면 된다.

(-128 ~ 0) : 내부가 찬 글자.
(0 ~ 127) : 외부만 차고 내부는 빈 글자.
(-128 ~ 30) : 조금 굵은 글자
(0 ~ 0) : 외곽선

해당 게임의 애니메이션은 Humanoid Retargeting 되어 모델로 들어가기 때문에, Animation 데이터가 LocalMatrix를 기준으로 구성될 필요가 있었고, 그것은 애니메이션을 계산할때 부모노드와 자식노드를 탐색하며 각 Bone의 WorldMatrix를 보여줄때마다 계산해야 하는 현상이 나타났다. 그로 인해 CPU 애니메이션을 할때 CPU 연산량이 많은 부분 소진되었고, fps가 제대로 나오지 못했다.

이를 해결하기 위해 GPU 애니메이션을 수행하기로 했다.
Humanoid Index, ModelNodeIndex, BoneIndex 간의 변환을 하면서
트리의 부모노드 탐색을 통해 최종 Bone의 World를 구하는 과정과
GPU로 애니메이션을 수행하고, 보여주는 것이 꽤나 나에게서는 난이도가 있었던 것
같다.



레이트레이싱

1. 레이트레이싱과 레스터에서 Mesh 정보를 공유

레이트레이싱 렌더링에서는 모든 버텍스/인덱스 데이터를 바인딩할 필요가 있었다. 왜냐하면 셰이더 Ray가 충돌한 지점의 평면의 노멀과 탄젠트, uv, 머터리얼 등의 정보가 필요했기 때문이다.

하지만 그렇다고 레스터와 레이트레이싱용 Mesh Res를 따로 만드는 것은 바람직하지 못했다. 왜냐하면 요새 GPU VRAM이 감소하는 추세이기도 하고, BLAS 빌드를 하고 나면, 남아있는 GPU 메모리가 부족할 것이기 때문이었다. 때문에 메쉬의 버텍스 데이터는 레스터와 레이트레이싱에서 같은 데이터를 사용해야 했다. 게다가 그런 데이터는 레스터에서의 VertexBufferView와 IndexBufferView를 만들 수 있도록 GPUVA가 정렬되어야 했기 때문에 새로운 메쉬데이터가 시작할 때마다 그 시작점을 정렬할 필요가 있었다. 정렬의 기준은 단일 버텍스의 사이즈와 256의 최소공배수가 되어야 할 필요가 있었기 때문에, 다음과 같이 버텍스 데이터를 잡았다.

48이 바람직했다. 768바이트 단위로 정렬된다.
extra + [LocalRootSig의 시작 머터리얼 인덱스]
를 하면 해당 점의 머터리얼 인덱스가 나온다.

```
//48byte (768)
struct Vertex
{
    float3 position; float u;
    float v; float3 normal;
    float3 tangent; uint extra;
};
```

2. 레이트레이싱 서브메쉬 처리 방법

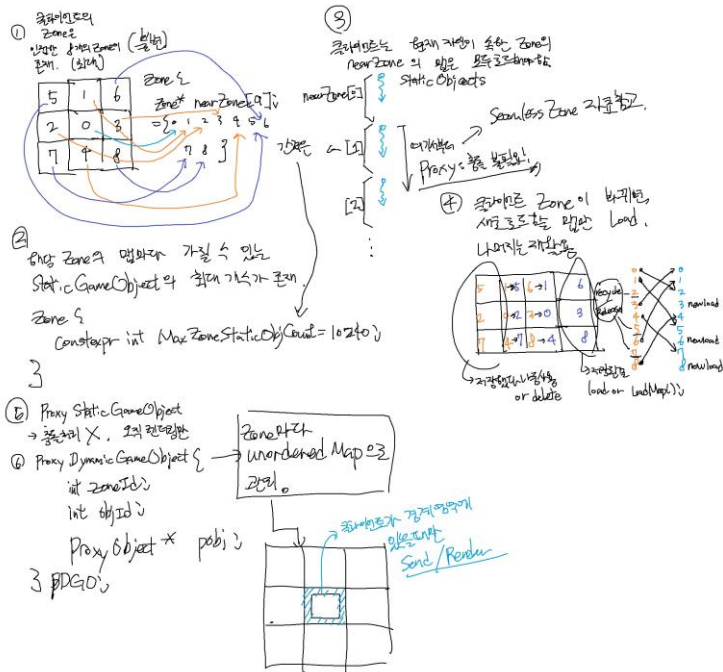
처음에 서브메쉬의 경우, Geometry와 BLAS를 서브메쉬마다 만들었었다. 그렇게 한 이유는 서로 다른 ShaderRecord를 써서 다른 머터리얼을 바인딩해야 했기 때문인데, fps가 아주 많이 떨어지는 사태가 발생했다. 이유는 서브메쉬들의 BLAS의 범위가 서로 겹쳐진 현상이 나타났고, 이 때문에 TraceRay가 ClosestHit를 찾는데 시간이 많이 소진되었다는걸 알았다. 그래서 ShaderRecord에는 머터리얼 시작 인덱스만 부여하고, 하나의 메쉬속 점들에서 각각 다른 머터리얼 인덱스를 가질 수 있도록 조정했다.

레이 트레이싱

3. 스킨메쉬 버텍스 모디파이 셰이더/BLAS Refit

레이트레이싱에서 스킨메쉬는 BLAS를 수정해야 했다 왜냐하면 해당 Geometry에서 Ray의 충돌여부 BLAS자체가 관여하기 때문이다. 다만 BLAS를 수정하려면 변경된 버텍스의 데이터가 필요했고, 때문에 스킨메쉬를 가진 오브젝트 하나당 스킨메쉬버텍스들을 따로 저장해야 했다.

때문에 UAV_VertexBuffer에 그런 스킨메쉬의 버텍스들을 모두 저장하고, 스킨메쉬 모디파이 셰이더를 만들고 사용해 버텍스 데이터를 수정하고, 그것을 통해 BLAS를 Refit 했다.

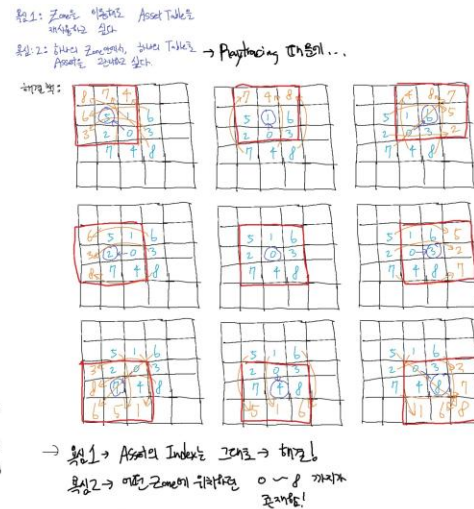


4. Seamless Zone을 Client에 적용시키기

심리스 존은 클라이언트가 어떤 존에 위치해 있으면, 인접한 존의 맵 데이터를 모조리 가지고 있어야 함.

문제는 존 데이터(머터리얼, 텍스처, 메쉬의 VB, IB, 스테틱 라이트, 청크와 GameObject들, 셰이더에 사용할 리소스 바인딩에 사용되는 DescHeap)들을 모조리 로드해야 하며, 이후 어떤 존으로 가도, 이후 존의 인접 존의 이미로드된 데이터는 유지를 해야 했다. 이를 위해 각 Zone에 특수한 인덱스인 AssetOffsetMul 을 만들었고, 해당 값만큼 이격된 위치에 모든 리소스가 GPU 에 존재하도록 만들었다. 그래서 Zone을 이동하더라도, 필요한 Zone의 리소스만 로드하게 만들었다.

TLAS Instance 또한 그냥 배열에 요소를 추가하는 방식이 아니라, 이전에 만들었던 BitAlloter를 사용하여 할당과 비할당처리를 해야 했다.



→ **목심 1** → Asset의 Index는 그대로 → 해결!
목심 2 → 어떤 Zone에 위치하면 0 ~ 8 까지
 존재함!

2. 양극 → ① 제1이 슬로

$$Arr = \begin{bmatrix} 5 & 1 & 6 \\ 2 & 0 & 3 \\ 7 & 4 & 8 \end{bmatrix} \quad Arr[x/3][y/3] = Arr_{xy} \quad result = \begin{bmatrix} Arr_{11} & Arr_{12} & Arr_{13} \\ Arr_{21} & Arr_{22} & Arr_{23} \\ Arr_{31} & Arr_{32} & Arr_{33} \end{bmatrix}$$

(2) zone에 보이는 불위를 끊어서 층수가 가능. ex) 5x5)

다중 ① 배역 Zone의 코드개별한 Asset의 개수 한계가 정해진다.

(2) 만약 zone의 모양이
Grid가 아닌 경우 적용이 매우 까다롭다

단순히 포인터로 이 작업을 할 수 없었던 이유는, Raytracing이나 Render에 사용되
는 많은 자료가 배열의 형태로만 저장될 수 있었기 때문이다.

예를 들어 Mesh의 VB, IB, Texture, Material의 DescHeap 내부의 View, 스킨메쉬버텍스 등등..

존을 이동했을때 유지되어야할 데이터는 놔두고, 업데이트할 데이터만 골라서 업데이트 할 수 있게 했다.

그 외의 기여

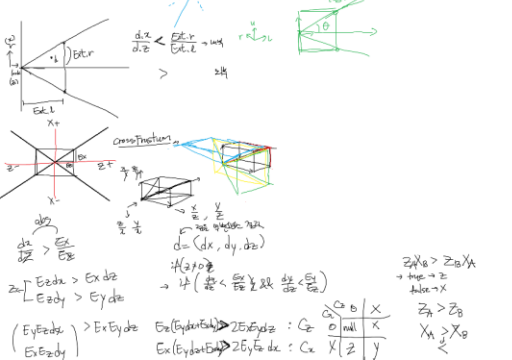
1. 필드의 모든 Zone 맵을 제작, 비주얼적으로 자연스럽게 연결함
2. 퀘스트 시스템과 퀘스트 데이터, 게임의 흐름을 구현.
3. 프로젝트가 추가적인 요구사항으로 변할때마다 합치거나 프로젝트 설계 자체를 다시 짜는 일을 모두,다수 수행
4. NPC의 움직임을 만들었다. NPC가 플레이어를 따라갈 수 있도록

원래 A*로 다른 팀원이 만든것이 있었는데, 너무 느린것 같아서 일단 비활성화 하고, 새로 만들었다.

5. 소통이나 정리를 위해 다수의 문서화를 진행했다.

ex> 우리 코드는 왜 더러운가, 그래픽스 하이레벨 디자인, 맵 제작 가이드라인, 모델 익스포트 가이드라인 등등..

6. FLStudio로 배경음악을 손수 제작했다.

[illegible][illegible]

$20V$

 $2A$

 $2V$

 $0.1V$

 $d = (5, 15, 25)$

 $E = (20, 0.5, 20)$

 $C = (0, 1, 0)$

 $E_x(E_y d_x) \Rightarrow 25 E_y d_x = 20$

 $E_x(E_y d_x) \Rightarrow 25 E_y d_x = 20$

 $E_y(E_x d_x) \Rightarrow 25 E_x d_y = 20$

 $20(0.5 \times 5 + 1.5 \times 20) > 2 \times 20 \times 0.5 \times 5 \times 25$

 $25 + 30 \quad \boxed{650 > 41}$

 $325 \rightarrow$

Diagram illustrating the geometric interpretation of the dot product. A vector p and a vector n are shown. The projection of p onto n is labeled $obt. uenkar. y = py$. The angle between p and n is labeled ϕ . The diagram also shows a vector n' and a vector p' .

$$A \cap \bar{B} = B \cap \bar{A}$$

정리 Center

Y마올려

3. 시온 포하시케리

$$\text{Get } y = \left(\frac{bx + cz + d}{b} \right)$$

plane $\vec{r} \cdot \frac{\vec{a}}{|\vec{a}|} \rightarrow p$
 $ax+by+cz+d=0$
 $\uparrow$ $\uparrow$
 $2\vec{a} \cdot \vec{b}$

두 물체가 서로 움직일때 충돌했다.
충돌 후 멈추게 된다면, 두 물체의
나중 위치는 어디인가?

Figure 1. Schematic representation of the experimental design. The top line shows the sequence of events for the first trial. The bottom line shows the sequence of events for the second trial. The red square indicates the start of the trial. The orange square indicates the start of the second trial. The green square indicates the start of the third trial. The blue square indicates the start of the fourth trial. The purple vertical line indicates the start of the fifth trial. The black circles indicate the end of the trial.

rate는 0.25다.

이름을 정한만 마면 빨간 지점에 도착하게 된다. > 도착 후 반역 소출한다던, 다시 뒤를회차 물리치
이름에 걸리는 정도를 rate만큼 곱다.

→ 0.5 - 0.25 = 0.25;

반역 소출하지 않는다면, 이름에 걸리는 정도를 rate만큼 곱한다.

→ 0.5 - 0.25 = 0.75;

그 rate를 반으로 나눈다.

이 과정을 반복하여 이진탐색을 한다.

만약 rate=length(vectory)/2 가 물리치 최소가되면 equal로 바로 찾아진다.

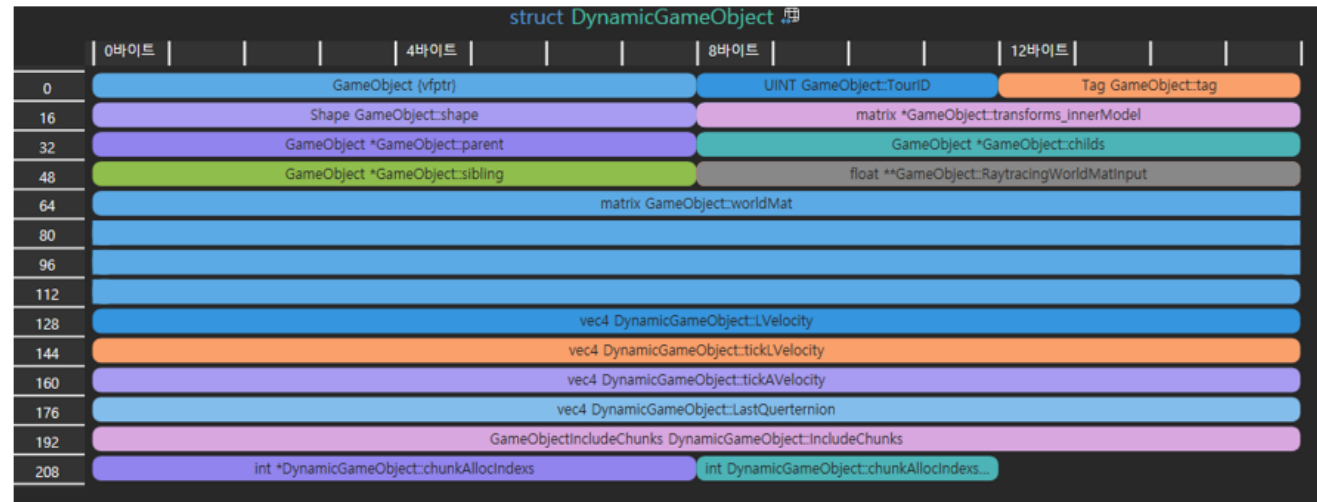
문제를 살펴보면, $vector[i] - min < vector[i] - max$ 를 수정한다.

```
epsilon = 0.01
length =
0.01 → 1번
0.02 → 2번
0.04 → 3번
0.08 → 4번
0.16 → 5번
```

부록 - 청크에 들어가기 위한 게임오브젝트 구조 설계

Chunk에 들어가기 위한 GameObject 구조

그것은 다음과 같은 GameObject 구조로 해결한다.



TourID : 해당 작업에 하나의 TourID를 할당받는다. 그리고 만약 해당 작업의 TourID와 오브젝트의 TourID가 다르다면 작업을 수행한 후 오브젝트의 TourID를 작업의 TourID로 바꾼다. 그러면 다른 청크에서 해당 오브젝트를 참조하더라도, TourID가 같으니 작업을 수행하지 않고, 결과적으로 어떤 작업이든 모든 오브젝트에 대해 한번씩 수행이 가능하다.

```
struct GameObjectIncludeChunks {
    int xmin;
    int ymin;
    int zmin;
    unsigned char xlen;
    unsigned char ylen;
    unsigned char zlen;
    unsigned char extraByte;
};
```

이동시의 문제는 각 축마다 최소와 길이를 가지는 범위와, 해당 범위에 청크들에서 해당 오브젝트가 어떤 인덱스에 있는지 저장하는 배열을 힙 메모리로 할당하여 해결한다.

```
GameObjectIncludeChunks IncludeChunks;
int* chunkAllocIndexs = nullptr;
int chunkAllocIndexsCapacity = 8;
```

Otornait

프로젝트 이름 : Otornait

장르 : 리듬게임

개발인원 : 2명 / 개발 기간 (2022-06 ~ 2026-07)

개발 환경 :

IDE/Debugger : VisualStudio2022

Language : c++

System API / Rendering : Windows API

Sound : Fmod

내가 담당한 역할 / 수행한 과제 요약

1. 게임 전체 프로그래밍 담당.
2. 게임플레이 화면, 타이틀화면, 평가화면, 크레딧, 레벨 에디터 등등
3. 리듬게임에 들어갈 곡 2개 제작, 레벨 2개 제작

게임 플레이 영상

https://www.youtube.com/playlist?list=PLjFfL6iibssVuoxZ-uO_7KU551bWSrDeT



그래픽스 기말과제

프로젝트 이름 : 그래픽스 기말과제

장르 : 어드벤처

개발인원 : 1명 / 개발 기간 (2022-11 ~ 2026-12)

개발 환경 :

IDE/Debugger : VisualStudio2022

Render : OpenGL, glsl

Language : c++ 14

System API : Windows API

Sound : Fmod

Resource Making : FL Studio, 3dsMax

Resource Finding : Sketchfab, IconFinder

Text Render : Freetype

본인이 과제를 모두 수행함.

과제 설명 영상

<https://youtu.be/XZY9EVAGHIk?si=QXp5YU6lOEM0dtPO>



InsideCode

프로젝트 이름 : InsideCode

프로그램 종류 : 바이트코드 컴파일러 / 바이트코드 실행기

개발인원 : 1명 / 개발 기간 (2023-03 ~ 2025-03)

개발 환경 :

IDE/Debugger : c4droid > Github Codespace > VisualStudio2022

Language : linux g++ > Microsoft msvc

본인이 모두 수행함.

이 프로젝트는 컴퓨터를 거의 할 수 없었던

병역의 의무를 질때 주로 수행되었습니다.

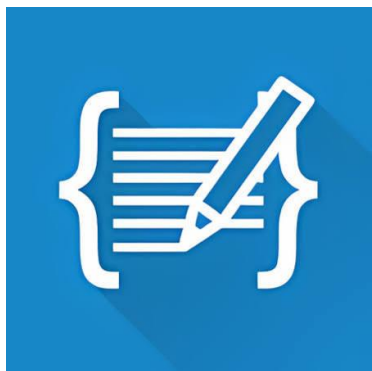
주로 일과가 끝나고 모바일 어플을(c4droid) 통해 개발했습니다.

상병이 되어서 그제서야 사지방에 갈 수 있었거든요.

사지방은 인터넷만 가능해 그때 Github Codespace로 작업을 시작했습니다.

환경이 이렇다 보니 제대로 개발하기가 힘들었어서 다소 오류가 존재할 수는 있습니다.

다만 학교를 졸업하고 취업을 하더라도 계속 수정할 저에게는 아주 소중한 프로젝트라 이렇게 소개합니다.



```
<nocode>
0      :FUNC(153)
1      :FUNCJMP(162) > 18(18, 0, 0, 0)

adsetvar: float PI = 3.141592
6      :PUSH_A_CONST_4(6) > 1078530008(216, 15, 73, 64)
11     :A=&Global_VID(178) > 0(0, 0, 0, 0)
16     :LA=A(161)
17     :*LA=A_4(171)

addfunc : int main ( )

adsetvar: vec2f pos1 = _vec2f ( 10 , 20 )
18     :ADD_STACK_VARIABLE(0) > 8(8, 0, 0, 0)
23     :FUNC(153)
24     :PUSH_A_CONST_4(6) > 20(20, 0, 0, 0)
29     :CASTING_A(163) > 12
31     :PARAM_4(156)
32     :PUSH_A_CONST_4(6) > 10(10, 0, 0, 0)
37     :CASTING_A(163) > 12
39     :PARAM_4(156)
40     :Extend_Instruction(255) > 0(0, 0, 0, 0) > 0(0, 0, 0, 0)
49     :A=&VID(159) > 8(8, 0, 0, 0)
54     :LA=A(161)
55     :*LA=A_N(182) > 8(8, 0, 0, 0)

adsetvar: vec2f pos2 = _vec2f ( 3 , 4 * ( pos1 . y - pos1 . x ) / 10 )
60     :ADD_STACK_VARIABLE(0) > 16(16, 0, 0, 0)
65     :FUNC(153)
66     :PUSH_A_CONST_4(6) > 4(4, 0, 0, 0)
71     :CASTING_A(163) > 12
73     :B<=&VID[Const](184) > 8(8, 0, 0, 0) > 4(4, 0, 0, 0)
82     :A=B(180)
83     :POP_B(175)
84     :A=*A_nopush(165)
85     :B<=&VID[Const](184) > 8(8, 0, 0, 0) > 0(0, 0, 0, 0)
94     :B=*B_nopush(166)
95     :AU_FLOAT_MIN_B(44)
96     :POP_A(174)
97     :AU_FLOAT_MUL_A(57)
98     :POP_B(175)
99     :PUSH_B_CONST_4(12) > 10(10, 0, 0, 0)
104    :CASTING_B(164) > 12
106    :AU_FLOAT_DIV_A(71)
107    :POP_B(175)
108    :PARAM_4(156)
109    :PUSH_A_CONST_4(6) > 3(3, 0, 0, 0)
114    :CASTING_A(163) > 12
116    :PARAM_4(156)
117    :Extend_Instruction(255) > 0(0, 0, 0, 0) > 0(0, 0, 0, 0)
126    :A=&VID(159) > 16(16, 0, 0, 0)
131    :LA=A(161)
132    :*LA=A_N(182) > 8(8, 0, 0, 0)

usefunc : dbg ( "pos1 : ( " , pos1 . x , " , " , pos1 . y , " )\n" )
137    :SET_A_CONST_STRING(172) > 9(9, 0, 0, 0)"pos1 : ( "
151    :DBG_A(167) > 8
153    :A<=&VID[Const](183) > 8(8, 0, 0, 0) > 0(0, 0, 0, 0)
162    :A=*A_nopush(165)
163    :DBG_A(167) > 6
165    :SET_A_CONST_STRING(172) > 3(3, 0, 0, 0)" , "
173    :DBG_A(167) > 8
175    :A<=&VID[Const](183) > 8(8, 0, 0, 0) > 4(4, 0, 0, 0)
184    :A=*A_nopush(165)
185    :DBG_A(167) > 6
187    :SET_A_CONST_STRING(172) > 4(4, 0, 0, 0)" )
"
196    :DBG_A(167) > 8
```


GMTK GameJam 참여작 - InverseProportion

프로젝트 이름 : InverseProportion

장르 : 플랫폼머

개발인원 : 1명 / 개발 기간 (2일)

개발 환경 :

IDE/Debugger : Unity, VisualStudio2019

본인이 과제를 모두 수행함.



게임 잼의 주제 : "Join Together" - 각기 다른 두가지 것이 연결되어 만들어지는 재미

>> 나의 기획 : 어떤 능력치의 반비례 관계를 활용한 플랫폼머 게임.

ex> 채력이 떨어지면 점프력이 올라간다! -> 닿지 않는 곳에 가기 위해 일부로 가시에 돌진

게임 플레이 : <https://nohc2001.itch.io/inverseproportion>

<웹에서 플레이하면 오류가 있습니다! exe로 다운받아 플레이를 해주세요!>

GMTKGameJam2021

https://www.youtube.com/watch?v=9U4Zoagd_40

GMTK GameJam 참여작 - GodChoice

프로젝트 이름 : GodChoice

장르 : 보드게임/전략

개발인원 : 1명 / 개발 기간 (2일)

개발 환경 :

IDE/Debugger : Unity, VisualStudio2019

본인이 과제를 모두 수행함.



게임 잼의 주제 : "Roll of the Dice"

- 운, 그리고 랜덤이 게임의 주요한 메카닉이 되어야 하지만, 운으로 결과가 결정되는 도박같은 게임이 되어서는 안된다. 운은 플레이어의 다양성과 지루함 극복을 위해 사용되어야 한다.

>> 나의 기획 : 1~6의 주사위를 굴려서 더 높은 사람이 공격을 함.
다만 매턴이 지날때마다 특정 차례의 주사위 눈 하나를 교환할 수 있다.
여기에 연속으로 같은 눈이 나오거나, 스트레이트가 될 경우
더 좋은 효과를 부여하는 것으로 전략을 짤 수 있도록 구성.

게임 소개 영상 : <https://youtu.be/qeZcDBkHkwv>

게임 플레이 : <https://nohc2001.itch.io/godchoice>

GMTKGameJam2021

https://www.youtube.com/watch?v=9U4Zoagd_40

